

COOKING ASSISTANT

Final Project Report

A fully offline mobile cooking guidance application

ICT 2140 – Introduction to Software Engineering
Group 14

Member	Registration number	Role
Ogechi Raïna Ornella	ICTU20251654	Scrum Master
Agwe Molly Trendy	ICTU20251685	Product Owner
Tebong Holician Endam	ICTU20251581	Team Member
Eteki Elame Dexter Kemmer	ICTU20251690	Lead Programmer

GitHub: <https://github.com/trendy23/COOKING-ASSISTANT->

Executive Summary

Cooking Assistant is a mobile app that helps people decide what to cook and how to cook it. The user enters the ingredients they have, the time they can spend, their energy level, their cultural preference and any dietary restrictions. The app removes unsuitable dishes, ranks the rest, and lets the user open a chosen recipe in an **Interactive Virtual Kitchen** that guides them step by step.

The app is built with Flutter and Dart and stores its data in SQLite on the phone. It needs no server or internet for its main features. Every user starts on the **Freemium** level. A redeem code unlocks **Premium** on the device. The project shows an offline-first design, simple rule-based recommendations, local access control and a beginner-friendly experience.

Purpose of this report: to describe the requirements, method, design, implementation, testing, design choices and team contributions of the finished project.

Chapter One – Introduction

1.1 General Introduction

Having ingredients at home does not tell a person what to cook or how to cook it. Cooking Assistant solves this by joining two things: recipe suggestions that fit the user, and a guided cooking experience. It is made especially for people with little or no cooking experience.

1.2 Problem Statement

A person may have ingredients, little time or energy, a cultural preference and dietary restrictions, and still not know what to cook. A beginner may also find a normal recipe hard to follow because it assumes cooking skills. Finally, apps that need the internet do not work well where the connection is weak or missing.

1.3 Aim

To build a fully offline mobile app that recommends suitable dishes from the user's inputs and guides the user through cooking with an Interactive Virtual Kitchen.

1.4 Objectives

- Recommend dishes from the available ingredients and personal limits.
- Consider time, energy level, cultural preference and dietary restrictions.
- Offer three recipe modes: Silent, Normal and Interactive.
- Guide the user step by step in the Interactive Virtual Kitchen.
- Let every user save favorite recipes, whatever their access level.
- Provide local Freemium and Premium access without outside login or payment.
- Keep all core features working without internet.

1.5 Scope

The project covers the local profile and access level, recipe and ingredient data, recommendations, recipe modes, Favorites, the Interactive Virtual Kitchen, and the local Freemium and Premium rules. It does not include a server, online accounts, payments or voice control.

Chapter Two – Key Concepts and Method

2.1 Key Concepts

Concept	Meaning in this project
Offline-first design	Core features work without a network. All needed data is stored on the phone.
Recommendation system	A system that selects and orders items for a user. Ours uses fixed rules (filter, match, rank), not machine learning.
Local database	SQLite stores recipes, ingredients, favorites and the access level on the device.
Freemium access	Free use with set limits. Premium removes the limits. Here, the level is checked on the phone.

2.2 Method

The team used Agile ideas with Scrum roles. The course asks for team organization, clear workflow and active GitHub work. The Scrum Master coordinates the process, so there is no separate traditional group leader.

Member	Role	Responsibility
Ogechi Raïna Ornella	Scrum Master	Testing, offline checks and build handling.
Agwe Molly Trendy	Product Owner	Requirements, matching rules, Free/Premium rules, recipe data and the Interactive Kitchen.
Tebong Holician Endam	Team Member	Walkthrough logic and Freemium/Premium limits.
Eteki Elame Dexter Kemmer	Lead Programmer	Database, business logic, matching algorithm and CRUD.

2.3 Team Workflow

Each member received a component, built it alone, reviewed the points where it joins other parts, pushed it to a branch, and then it was reviewed and merged.

Assign component → Build → Review links → Push branch → Merge after review

2.4 Why Agile and Scrum

The project has several modules that can be built at the same time. Agile and Scrum allow this split while keeping the team focused on how the modules fit together.

Chapter Three – Requirements and System Design

3.1 Actors

Actor	Description
Freemium User	The default local user. Gets the core features with set limits on recommendations and the Virtual Kitchen.
Premium User	A user who unlocked Premium locally. Gets unlimited recommendations and the full Virtual Kitchen.

3.2 Functional Requirements

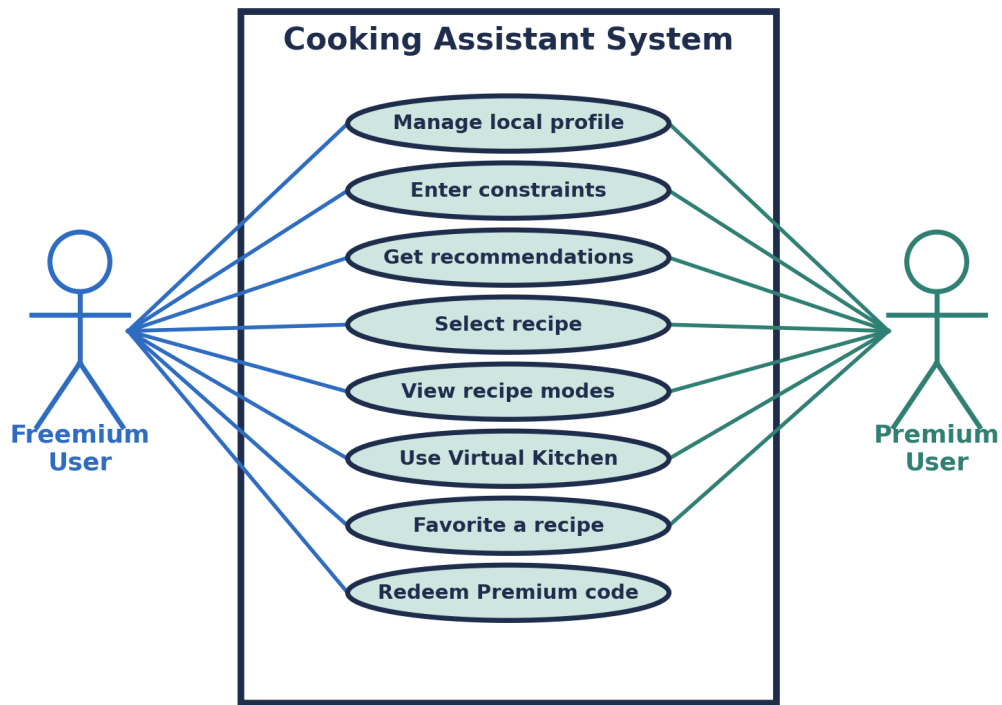
ID	Requirement
FR1	The system shall let a user create or select a local profile without outside login.
FR2	The system shall accept ingredients, time, energy level, cultural preference and dietary restrictions, and return recommended dishes.
FR3	The system shall let a user select a recommended dish and enter the Interactive Virtual Kitchen.
FR4	The system shall let users save favorite recipes at any access level.
FR5	The system shall offer Silent, Normal and Interactive recipe modes.
FR6	The system shall let a Freemium user unlock Premium locally with a valid redeem code.
FR7	The system shall cap Freemium recommendation requests. Premium users have no cap.
FR8	The system shall limit Freemium Interactive Kitchen access to starter recipes. Premium users can use all recipes.
FR9	The system shall keep Favorites available to both Freemium and Premium users.

3.3 Non-Functional Requirements

ID	Requirement	Purpose
NFR1	Fully offline operation	No network or server needed.
NFR2	Recommendations within 2 seconds	A fast, responsive app.
NFR3	Simple and clear screens	Easy for users with no cooking or technical experience.

3.4 Use Case Diagram

The diagram shows what each user can do. Both users can use every feature, but Freemium users meet the limits shown in orange. Only a Freemium user needs to redeem a code.



Blue lines: Freemium user. Green lines: Premium user.

Freemium limits: capped recommendations, starter Kitchen recipes only.

Figure 1. Main user interactions with the Cooking Assistant system.

3.5 Class Diagram

The diagram shows the main data classes (green and navy) and the service classes (orange). Arrows show which class uses or controls another.

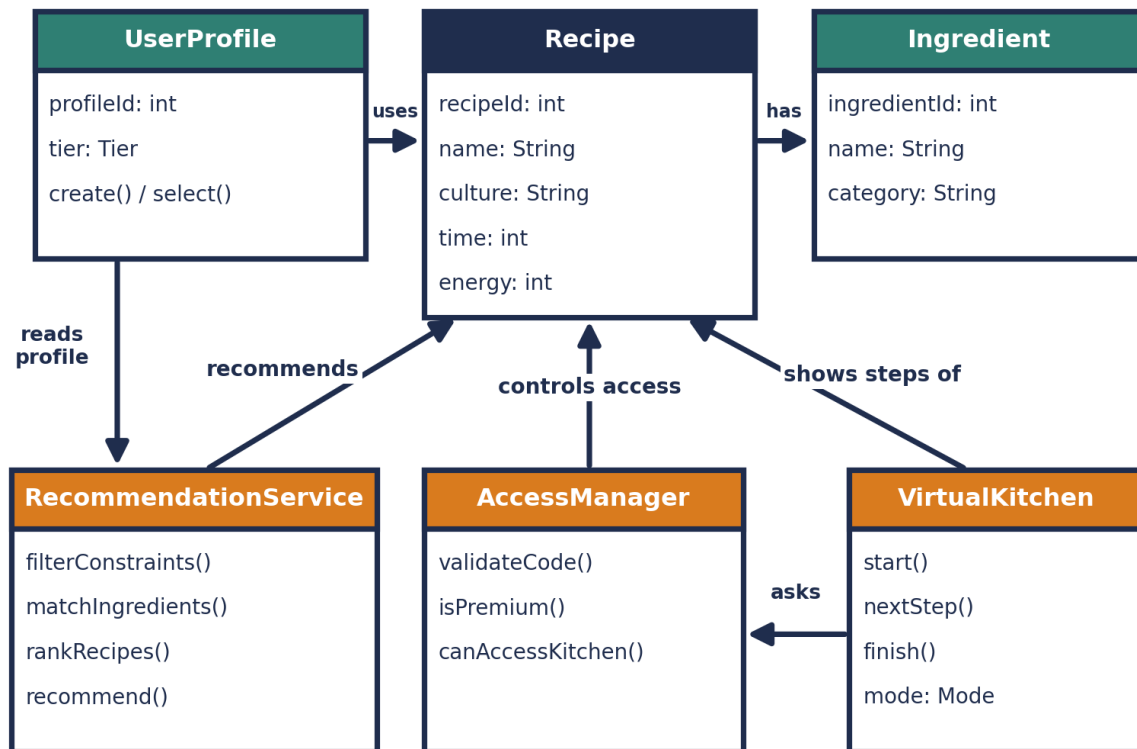


Figure 2. Core classes and how they work together.

3.6 High-Level Architecture

The app has three layers. Each layer talks only to the one next to it.

Layer	Parts	Job
Presentation	Flutter screens: Home, Recipe, Favorites, Virtual Kitchen	Collect input and show results.
Application logic	RecommendationService, AccessManager, VirtualKitchen	Apply the rules and coordinate features.
Data	SQLite: recipes, ingredients, favorites, access level	Store data on the phone.

3.7 Core Recommendation Algorithm

1. **Collect** the ingredients and the user's limits.
2. **Filter** out dishes that break fixed rules, such as dietary restrictions.
3. **Match** the remaining dishes with the ingredients available.
4. **Rank** the matches using softer wishes: time, energy and culture.
5. **Return** the best matching dishes.

3.8 Sequence Diagrams

Each diagram reads from top to bottom. Solid arrows are requests. Dashed green arrows are replies. A loop arrow means the part is working on its own.

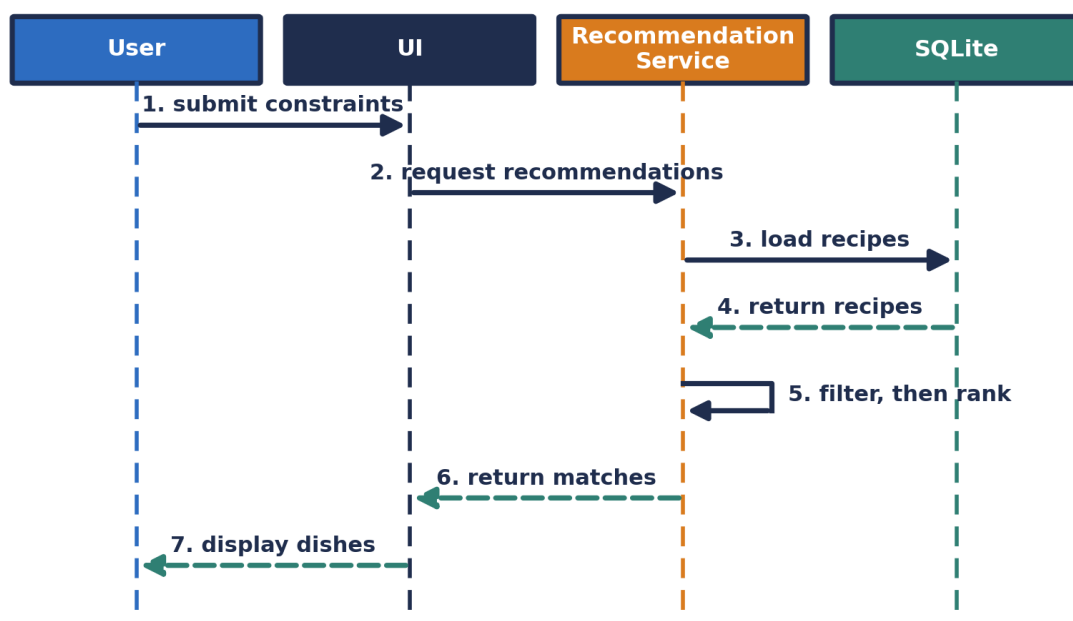


Figure 3. Getting recommendations.

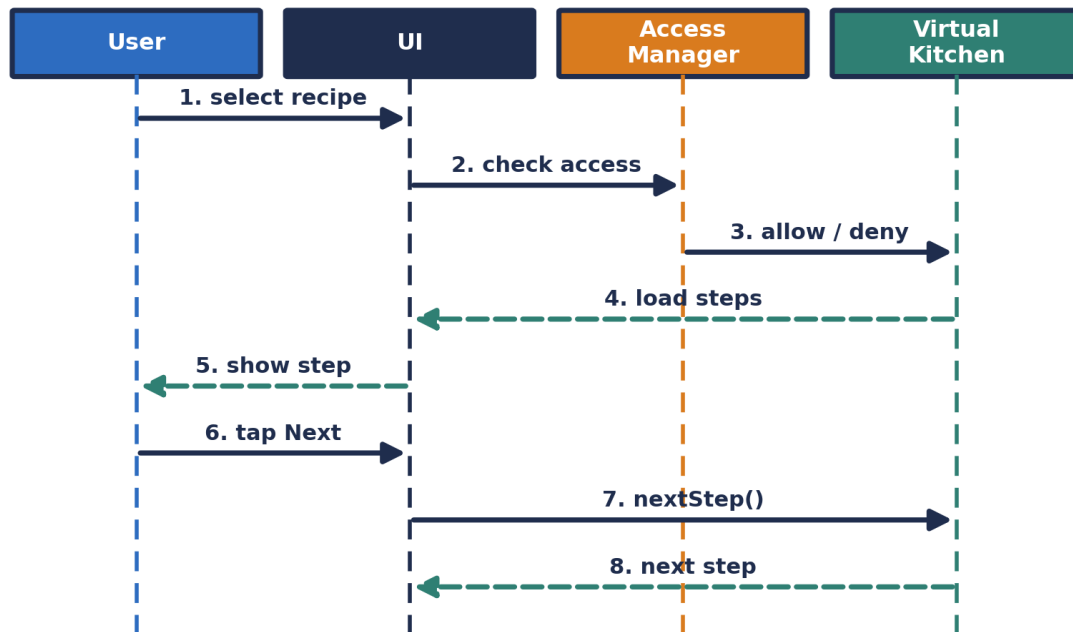


Figure 4. Opening the Interactive Virtual Kitchen. The Access Manager decides who may enter.

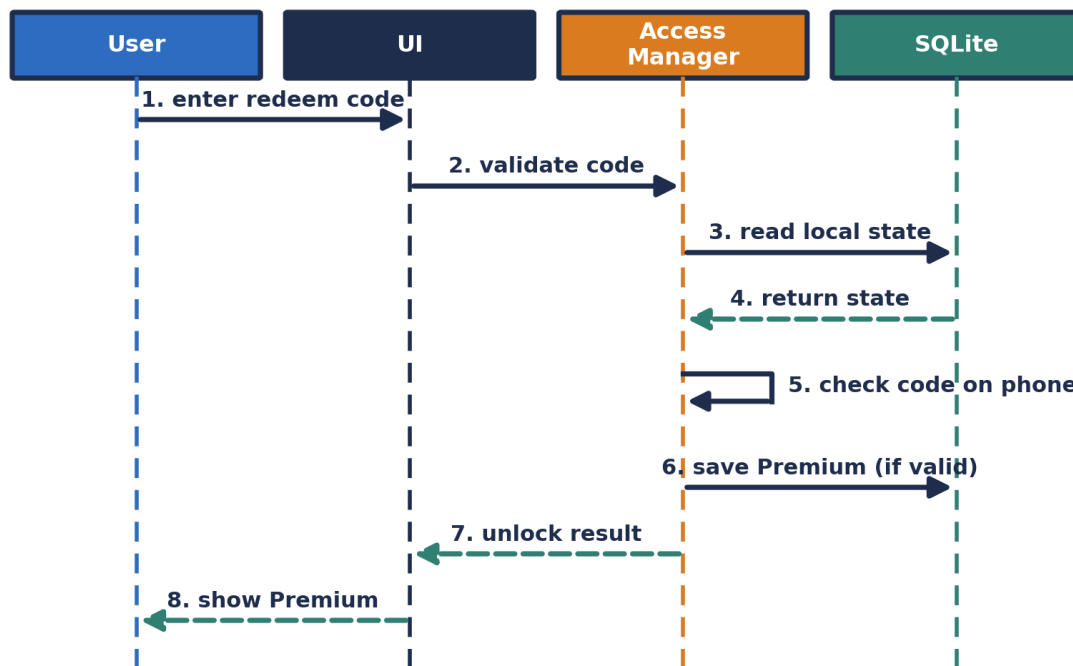


Figure 5. Redeeming a Premium code. The code is checked on the phone.

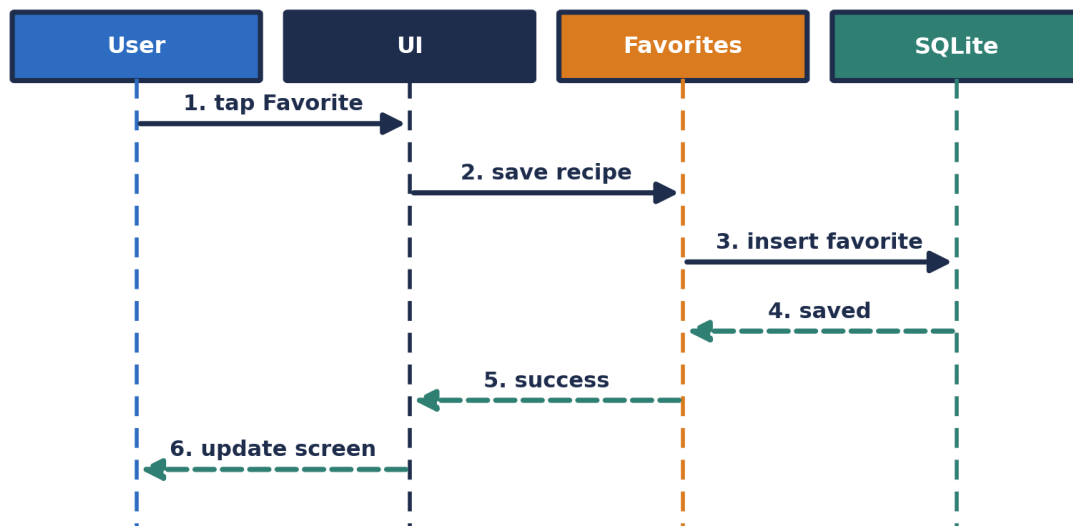


Figure 6. Saving a favorite recipe.

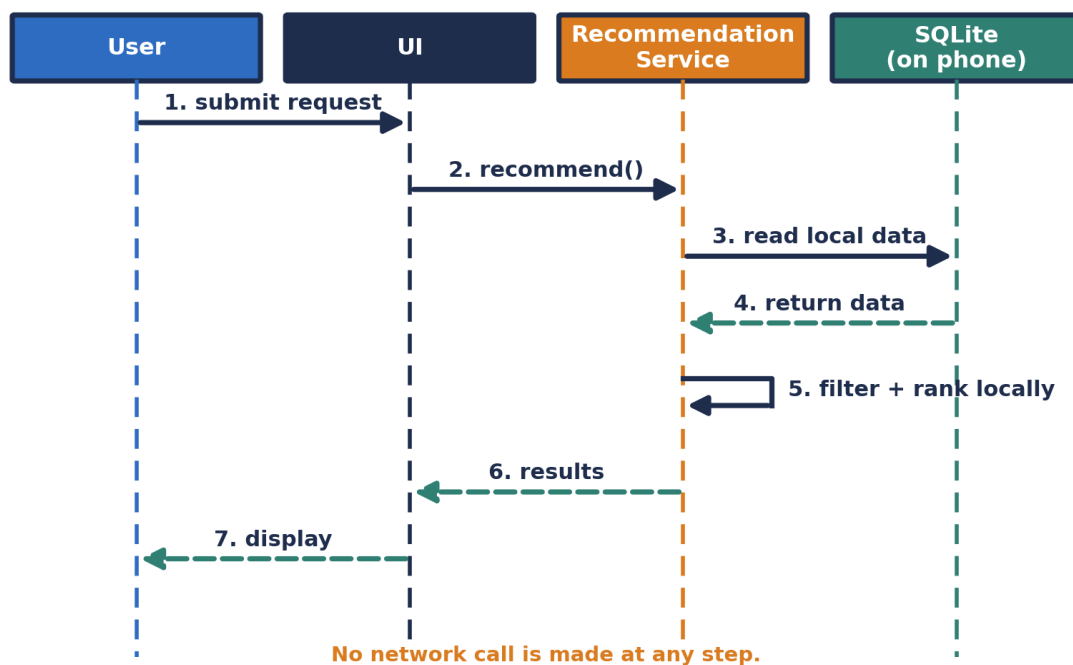


Figure 7. Offline recommendation. Everything happens on the phone.

Chapter Four – Implementation, Testing and Results

4.1 Technology Stack

Technology	Use
Flutter	Mobile screens and app packaging.
Dart	App logic and business rules.
SQLite	Local offline database.
GitHub	Version control and teamwork on branches.

4.2 Main Modules

- Local profile and access level
- Ingredient and constraint input
- Recommendation engine
- Recipe display modes
- Interactive Virtual Kitchen
- Favorites
- Freemium and Premium access control
- Local recipe and ingredient database

4.3 Testing Strategy

Testing checks the behavior users can see and that the app works offline. The course allows white-box testing, black-box testing or both. Test evidence should be shown for each main requirement.

ID	Test	Expected result
TC01	Valid ingredients and constraints	Suitable recommendations are returned.
TC02	Dietary restriction conflict	Conflicting recipes are removed.
TC03	Freemium request limit	The limit is enforced after the set cap.
TC04	Premium redeem code	A valid code unlocks Premium locally.
TC05	Starter Virtual Kitchen	A Freemium user can open an allowed starter recipe.
TC06	Restricted Virtual Kitchen	A Freemium user cannot open a Premium-only walkthrough.
TC07	Favorite a recipe	The recipe is saved and appears in Favorites.

ID	Test	Expected result
TC08	Offline operation	Core functions keep working without a network.
TC09	Recipe modes	Silent, Normal and Interactive modes open correctly.
TC10	Performance	Recommendations appear within 2 seconds.

4.4 Results

The project status is **complete**. The app has all the core features listed in this report. For the final presentation, real app screenshots and a live demonstration should be shown as proof.

4.5 Design Choices and Trade-offs

Decision	Reason and effect
No backend or server	Keeps the app fully offline.
Rule-based recommendations	Same input gives the same result. Easy to explain and test.
Local Premium redeem code	Works offline. It is not built to resist tampering on the device.
Hands-free voice left out	Avoids a half-finished voice or AI feature.

4.6 Challenges and Responses

Challenge	Response
Working offline	Local SQLite database with built-in recipe data.
Complex recommendations	Filter first, then rank.
Access levels without a server	Redeem codes checked on the device.
Advanced hands-free feature	Left out to protect core reliability.
Joining the team's work	Branch-based GitHub workflow with reviews.

Chapter Five – Recommendations and Conclusion

5.1 Recommendations

- Explore reliable offline voice control.
- Use stronger Premium checks if an online model is added later.
- Add more recipes and more cultures.
- Add personal suggestions that protect privacy.
- Add more accessibility options, such as audio guidance and larger text.

5.2 Conclusion

Cooking Assistant is a complete software engineering answer to an everyday cooking problem. It links the user's limits to suitable dishes and then gives a clear path for cooking them.

The team chose to put a working offline core first. The architecture, rule-based algorithm, local access model and Agile teamwork were all chosen for that goal. The app can later grow with richer accessibility, more content and optional advanced features.

Appendices

Appendix A – Requirements Traceability

Requirement	Where it is built
FR1	Local profile and access level
FR2	Ingredient and constraint input, RecommendationService
FR3	Interactive Virtual Kitchen
FR4	Favorites
FR5	Recipe modes
FR6	AccessManager and redeem code
FR7	Recommendation request limit
FR8	Virtual Kitchen access by level
FR9	Favorites for both levels
NFR1	SQLite and built-in local content
NFR2	Performance testing
NFR3	Beginner-focused Flutter screens

Appendix B – Presentation Demonstration Checklist

- Open the app and show the default Freemium state.
- Enter ingredients and constraints.
- Show the recommendations.
- Open a recipe in the Interactive Virtual Kitchen.
- Show the recipe modes.
- Show Favorites.
- Show the Premium redeem flow, if suitable.
- Show GitHub branches and contribution history.
- Summarize the architecture and the problem solved.

Appendix C – Team Contributions

Member	Contribution
Ogechi Raina Ornella	Testing, offline checks and build coordination. Scrum Master.
Agwe Molly Trendy	Requirements, product rules, recipe data and Interactive Kitchen. Product Owner.
Tebong Holician Endam	Walkthrough logic and access-level limits.
Eteki Elame Dexter Kemmer	Database, business logic, matching algorithm and CRUD. Lead Programmer.